Intrusion Detection Systems: Analyzing Network Security Threats

Devi Sree Vipperla Bharadwaj Gutha Satwika Davala Hasmitha Krishna Nunna

I. PROBLEM STATEMENT

Enhanced with digital capabilities over the past decade particularly on internet, there has been cyberattacks posing risks to organizations as well as individuals. The importance of this issue is rooted in the increasing tendency of cyber-criminals attempting to exploit network vulnerabilities to get unauthorized access to personalized and sensitive information compromising services and cause financial loss. Traditional intrusion detection systems IDS that often depends on predefined rules and signatures have their limitations in detecting dynamic and modern forms of attacks. As such, this project aims at the general goal of advancing IDS by employing modern Machine learning approaches to improve the detection capabilities of Intrusion detection system.

II. BACKGROUND OF THE PROBLEM AND OBJECTIVES

Computer and network vulnerabilities are perceived to be intensifying with enhanced cyber attacks on data and communication systems. This is a problem because conventional IDS procedures struggle to detect advanced evolving attack patterns, often missing modern subtle threats. Because of important work that networked systems do, for example, in finance, health care and government services, accurate real-time intrusion detection is a greater concern than ever. The UNSW-NB15 dataset was specifically chosen for this project due to its realistic representation of modern network traffic and diverse attack types, making it an essential benchmark for intrusion detection systems.

III. CONTIRBUTION POTENTIAL AND ITS IMPORTANCE

The goal of this project is to assist network administrators and organizations in detecting and preventing cyberattacks using deep learning-based IDS. By providing valuable insights into the techniques and strategies for implementing these systems, administrators can better protect their networks and data from malicious actors. One of the key benefits of deep learning-based IDS is their ability to adapt to new and evolving threats. Additionally, deep learning-based IDS can analyze large volumes of network traffic quickly and accurately, allowing administrators to respond to potential threats in real time. By providing early detection of cyber-attacks, these systems can help organizations mitigate the impact of data breaches and prevent sensitive information from falling into the wrong hands. Overall, the implementation of deep learning-based IDS represents a significant step forward in enhancing cybersecurity for organizations. By leveraging the power of artificial intelligence, administrators can gain valuable insights into detecting and preventing cyberattacks, ultimately reducing the risk of data breaches and protecting critical network infrastructure.

IV. DATA SOURCES

Dataset: UNSW-NB15 Dataset on Kaggle

The dataset which is adopted in this work is the UNSW-NB15, well recognized as a computer network security dataset released in 2015. It is comprised 175,341 realistic premises of network behavior and includes network activity that may be regarded as normal or abnormal. CoA was designed within the framework of a research project at University of New South Wales (UNSW) to improve the assessment of network intrusion detection systems (NIDS). Flow-based and packet-based information is included in the dataset which contains 45 attributes derived from raw network packets. The set of labels consists of traffic categories between normal and different kinds of attacks like Fuzzers, Analysis, Backdoors, Denial of Service (DoS), Exploits, Generic, Reconnaissance, Shell code, and Worms. This is a common set of attributes which gives detailed information about the packet and byte counts all of which are very important for traffic analysis. However, it has more than 175000 records which provides a wide dataset for some deeper analysis and model training.

Link for the dataset: UNSW-NB15 Dataset on Kaggle

V. DATA CLEANING/PRE-PROCESSING

Before analyzing, the given data set was cleaned to ensure that it meet the standards of machine learning models input. There were several basic operations made about missing data, remove duplicates, correcting the inaccuracies, managing outliers and making it ready for further analysis

A. Handling missing values

Missing values can lead to skewed analysis and model performance. There are some instances of '-' in the `service` column that indicate missing or irrelevant values, which were then replaced with `NaN`. In the following steps, we want to apply imputation or removal strategies, and for that, we have to put placeholders `NaN` appropriately.

B. Remove Duplicates

Duplicates should be removed as it can affect the model results and analysis. The dataset taken does not contain duplicates, so removing them for the provided dataset is not required.

C. Dropping Irrelevant Columns

The `service` column was seen to be closely related to the identification of the least relevance to analytical goals of the project. Thus, it has been eliminated from the dataset. When there is an issue of a large number of features and some are not going to be relevant in the model, it is much easier to deal with such data. Pre-processing techniques could therefore help in improving the accuracy of the model. This has a beneficial impact on the criterion because it excludes unimportant variables that waste time to analyze in that stage.

D. Outlier Detection and Removal

Outliers can often tend to influence statistics and machine learning more than other data points. To address this, the Interquartile Range (IQR) method was used to identify and remove outliers from numerical columns such as `spkts`, `dpkts`, `sbytes`, `dbytes`, and `rate`. Extreme values are created through outliers, which contaminate statistics and cause models to overfit. When cleaning data, such outliers are to be dealt with so that the distribution gets a proper representation and improves the model's reliability.

E. Feature Engineering

To increase the dataset's usability for analysis, a new variable, `pkt_byte_ratio`, was derived, which indicated the flow's packets-to-bytes ratio. Feature engineering should always be used in analysis to maximize model accuracy and enhance its interpretability. Another value, which illustrates network performance, can also influence intrusion detection analysis – the packet-to-byte ratio.

F. Encoding Categorical Variables

Categorical variables were recoded into numerical form and made available to fit most of the machine learning algorithms. The `label` column was encoded using `LabelEncoder` and the `attack_cat` column was One-Hot Encoded. In reality, most of the machine learning algorithms are operationally inclined toward numerical input. Categorical variables need to be encoded because including them in the analysis will help improve the results of the model within the dataset.

G. Data Normalization

Finally, numerical features were scaled, making all values within the proper range for algorithms that are sensitive to feature scaling. Normalization stretches the range of feature values to the range [0, 1], a process that is very useful for distance-based modeling such as K-NN classification and also for optimizing models using gradient descent. This step ensures that features with large ranges do not unnecessarily suppress model training.

VI. EXPLORATORY DATA ANALYSIS (EDA)

Exploratory Data Analysis (EDA) is used to understand the structure and characteristics of a dataset before modeling. It helps detect patterns, trends, and anomalies, identify data quality issue. EDA ensuring better model performance and more accurate decision-making.

A. Descriptive Statistics

The `describe()` function was utilized to compute summary statistics [Fig. 1], focusing on central tendencies and dispersions of key numerical variables such as `spkts`, `dpkts`, and `sbytes`. These insights revealed trends and helped detect the presence of outliers.

Outcome: The large variability in packet counts and byte sizes, reflected by high standard deviations, indicates substantial differences in network traffic behavior, potentially signaling diverse network activities or malicious events.

	id	dur	spkts	dpkts	sbytes	dbytes	rate	sttl	dttl	slo
count	175341.000000	1.753410e+05	175341.000000	175341.000000	175341.000000	175341.000000	175341.000000	175341.000000	175341.000000	175341.0000
mean	0.500000	2.265646e-02	0.002007	0.001729	0.000680	0.001019	0.095406	0.704106	0.313423	0.0122
std	0.288678	1.080042e-01	0.014237	0.010047	0.013480	0.009802	0.165401	0.403686	0.435066	0.0314
min	0.000000	0.000000e+00	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.0000
25%	0.250000	1.333334e-07	0.000104	0.000000	0.000007	0.000000	0.000033	0.243137	0.000000	0.0000
50%	0.500000	2.636667e-05	0.000104	0.000182	0.000031	0.000011	0.003226	0.996078	0.114173	0.0001
75%	0.750000	1.113449e-02	0.001144	0.000911	0.000107	0.000075	0.125000	0.996078	0.992126	0.0148
max	1.000000	1.000000e+00	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.0000

Fig. 1: Descriptive Statistics

B. Visualization of Distribution of Normal and Abnormal Attacks

Observation: In this part, we will look at the distribution of normal and abnormal attacks. This allows us to determine whether the dataset has any biases that have a major influence on performance. To visualize, we developed a pie chart[Fig. 2] comparing the proportion of normal and anomalous traffic. Understanding the distribution is extremely useful in assessing the necessity for approaches such as class balance throughout the training phase. Even though this binary classification helps distinguish between normal and abnormal attacks.In this project, we aim doing multi classification to identify specific attack types to accurately detect the nature of each attack. Now, we present a distribution of various attack categories[Fig. 4] using a pie chart[Fig. 3].

Usage of EDA: This EDA step allows us to understand the distribution of classes and check for class imbalance between the normal class and various types of attacks, which may affect the model's performance.

Outcome: The pie chart shows the distribution of normal traffic and nine different types of attacks, which helps to determine further actions such as class balancing and improving the multi-class model.

C. Correlation Analysis

Observation: A correlation matrix was computed to identify relationships between numerical features. It is also evident that features such as sbytes and dbytes are strongly related to each other as are spkts and dpkts. Some features are almost uncorrelated with others, such as id [Fig. 5].

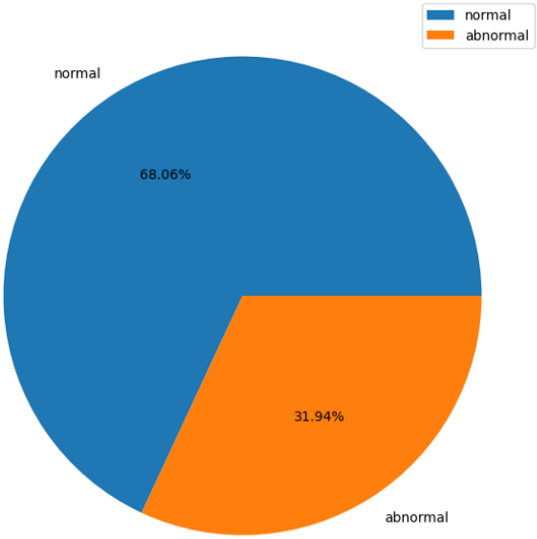

Fig. 2: Pie chart visualization of normal and abnormal labels

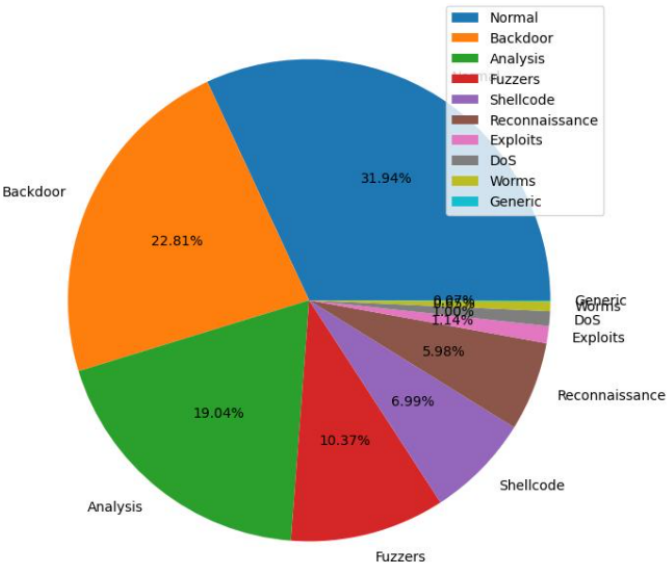

Fig. 3: Pie chart distribution of multi-class labels

Usage of EDA: The matrix shows crucial relationships between different variables. Strong correlations suggest key areas to investigate further. In contrast, weak correlations indicate that those relationships are not very significant. Knowing these relationships Can help improve the detection of malicious activities.

Outcome: The correlation matrix is much more informative in terms of feature dependencies and helps to understand how various network traffic parameters are engaged. This particularly is useful when developing the machine learning or deep learning models by explaining the features dependence.

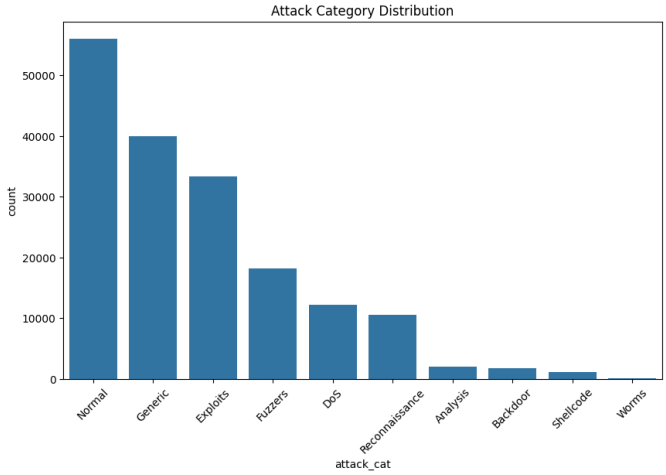

Fig. 4: Attack Category Distribution

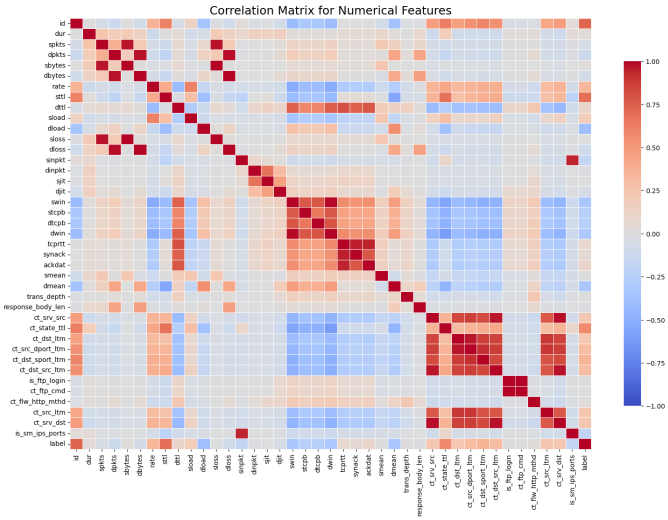

Fig. 5: Correlation matrix for numerical features

D. Box Plot

Observation: The box plots show the distribution of source packets (spkts)[Fig. 6] and destination packets (dpkts)[Fig. 7] for various attack types. The packet counts in categories such as Exploits and Denial of Service (DoS) have a larger spread with outliers clearly visible beyond the upper quartile. However, Backdoor, Analysis, and Worms categories show less and more focused packet distribution, which means less packet activity.

Usage of EDA: These box plots give an overview of the distribution of packet counts per attack type, and the measures of central tendency, dispersion, and skewness for each category. This assists us in determining the attack types that produce more traffic and the attack types that have more constant packet counts.

Outcome: These plots show that DoS and Exploits have a much higher packet traffic and therefore packet count is a good feature for these attacks. This insight is useful in identifying

features for detection, enhancing the performance of IDSs by concentrating on traffic intensive attacks.

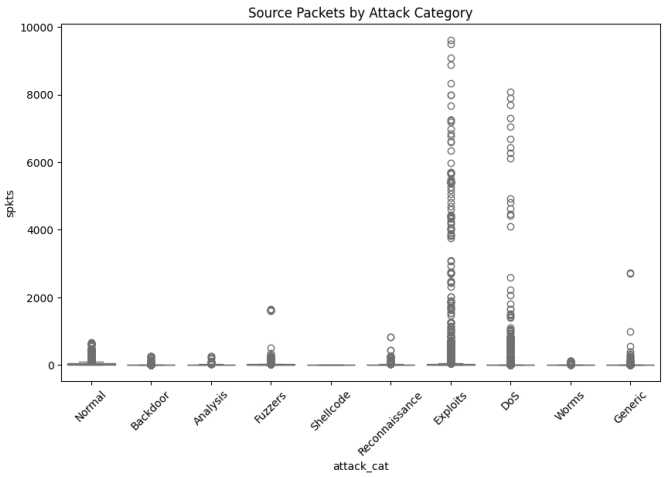

Fig. 6: Source Packets by Attack Category

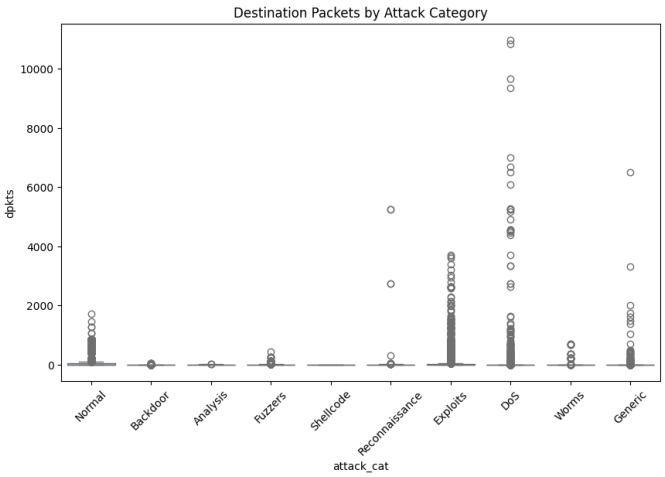

Fig. 7: Destination Packets by Attack Category

E. Violin Plot

Observation: Each of the attack categories has different distributions, implying different rate variations. The median rates indicate which types of attack are higher or lower. As to the width, it points to density; the higher the width of the bar, the more often the rates were. The extended sections point out values that should be studied as potential outliers. Different colored violins denote varied properties of each attack kind as shown in [Fig. 8]

Usage of EDA: Assists analysts to understand the frequency as well as the level of attack instances. Anomaly Detection like observing such patterns can be useful in further analysis that can help spot any serious abnormal increase. Makes it easier to identify what needs to be fortified when it comes to particular category of attacks.

Outcome: Contributes to establishing understanding of security threats and determining the directions of their counter-action. Trends are defined to identify what the practice should aim for. Transmits information that can include many variables and variables that are important in the decision making process to stakeholders.

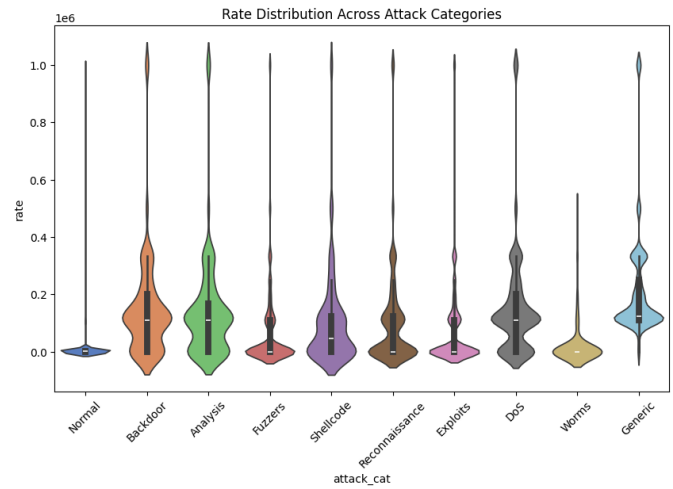

Fig. 8: Violin plot for Rate Distribution across Attack Categories

F. Multivariate Analysis

Observation: The pairplot visualizes relationships between key numerical variables (`spkts`, `dpkts`, `sbytes`, `dbytes`, `rate`) across different attack categories (e.g., `Normal`, `Backdoor`, `DoS`). The color-coding by `attack_cat` shows clusters and patterns of network traffic across various attack types[Fig. 9].

Usage of EDA: This multivariate analysis using pairplots helps in understanding relationships between numerical variables and attack categories. It helps us to view the distribution of features in each attack category and identify overlapping patterns between categories

Outcome: The pairplot explains perticular attacks , like `DoS` and `Shellcode`, show unique patterns in packet and byte counts compared to normal traffic. For instance, the `DoS` category seems to have much larger values for `sbytes` and `dbytes`. This visualization provides information on how specific attacks differ from normal behavior, so these features are useful for classification models in detecting different attack types.

G. Principal Component Analysis

Observation: The PCA scatter plot visualizes the data set in to 2 different components (`pca1` and `pca2`), so we can view the spread of data points across different attacks (e.g., `Normal`, `Backdoor`, `DoS`) as shown in [Fig. 10]. We can observe that the data points for some categories seem to be clustred and overlap each other , which tells us that they have very similar data in feature space.

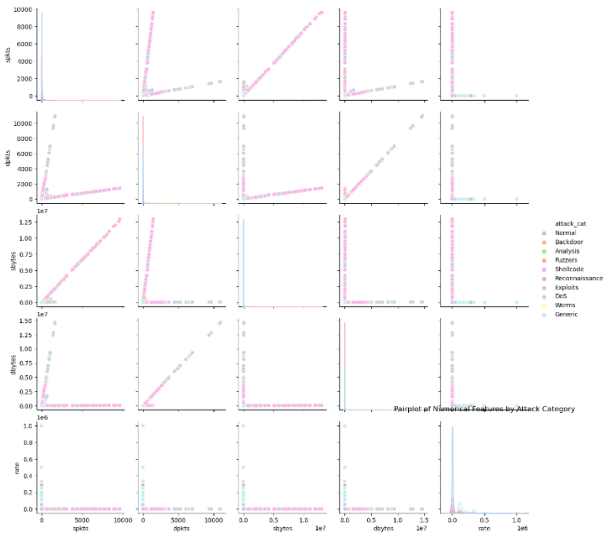

Fig. 9: Multivariate Analysis

Usage of EDA: Principal Component Analysis (PCA) is used to reduce the dimensionality of the dataset, helping to user to find out patterns or clusters in the data. This method reduces the data set into small component which keeping as much variance as possible, resulting in visualizing how different attack types relate to each other in samller dimensions.

Outcome: The PCA plot tells us that particular attack categories, like `DoS` and `Fuzzers`, show unqiu patterns in smaller feature space, while others features , such as `Normal` and `Generic`, show overlapping . we can tell that while PCA can separate some attack types but not all.

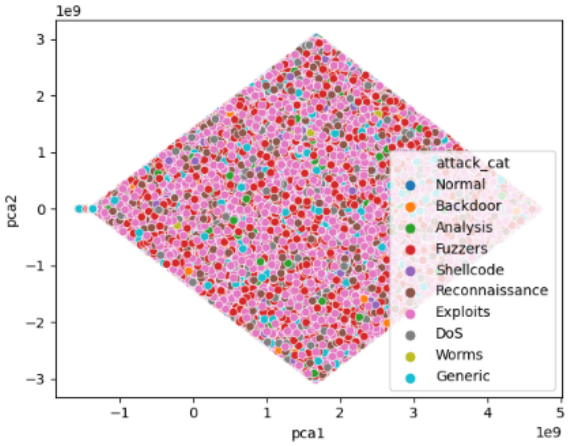

Fig. 10: Principal Component Analysis

H. Feature Engineering

Observation: A new feature, `pkt_byte_ratio`, has been added by calculating the ratio of packets (`spkts`) to bytes (`sbytes`). This feature can provide information about the average size of the packets transmitted during network flows.

Usage of Feature Engineering: By adding the new feature `pkt_byte_ratio`, we can improve the data knowledge by getting to know the relationship between the number of packets and the amount of data transmitted. This ratio can be used to detect traffic anomalies,as a certain attack might have a different packet/byte ratio

Outcome: The new feature, `pkt_byte_ratio`, can improve the overall analysis providing a higher level details to traffic behaviors. For instance, traffic with many packets but little bytes can show certain sorts of attacks, such as denial of service). This feature proves useful to models that use traffic flows for differentiation or to identify distinct sort of attack.

I. Anomaly Detection

Observation: The Isolation Forest model was used to find the anomalies in the dataset, which marked around 10% of the data points as anomalies (anomaly score of `-1`). The scatterplot between `spkts` (source packets) and `dpkts` (destination packets) shows the anomalous points, and we can see a few data points away from the main cluster[Fig. 11].

Usage of Anomaly Detection: Isolation Forest is used to detect anomalies by identifying abnormal patterns in the numerical features of the dataset. The model will isolate the data points which are away from the main cluster. The identified anomalies are viewed in a scatterplot.

Outcome: We successfully detect the outliers in the dataset ,mainly those data points, where the source and destination packets show unusual behavior to the main data points . These anomalies can be assumed to be a malicious activity or network attack, so we can classify them and investigate the flagged network transaction.

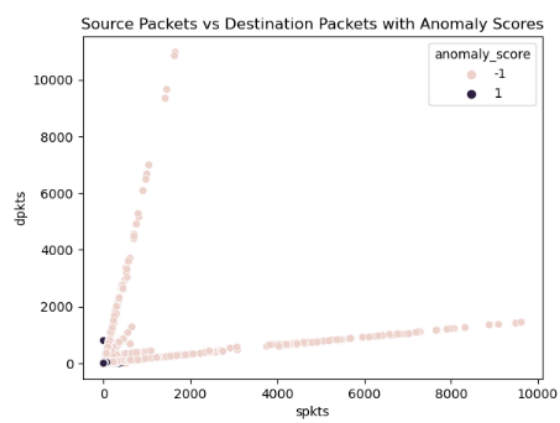

Fig. 11: Anomaly Detection

VII. MODEL SELECTION AND EVALUATION

A. *Linear Regression*

- **Justification** : Linear regression is a model which works the best when relations between independent variables (features) and the dependent variable (target) is linear, This model is very simple and interpretable, linear regression allows you to directly assess how each feature

influences the predicted value.. And linear regression is used a baseline model to be compared to other models.

- **Training Details** : The LinearRegression model was fit on the binary dataset using feature Set= X_train_binary and label Set= y_train_binary. The fit method was applied to understand the nature of direct mapping between the inputs and outcome data.
- **Prediction** : After training the model, the model was used to make prediction for the test dataset: X_test_binary. The predicted output is continuous, which expresses the probability of the positive class where class 1 is a positive class.
- **Evaluation and Tuning** : Hyperparameters doesn't need to be tuned as much in binary classification when using linear regression. However it is important to evaluate through metrics such as accuracy, precision, and recall in other to know how efficient the model is in predicting the binary values. The thresholding function may also be tweaked from the specifics of a need or application environment.
- **Insights** : The plot proves the efficient operations of the model in a graphical way. We also see that the predicted values are quite similar to the actual values, which means that our classes have been predicted quite correctly. Finally, quantitatively the model has an accuracy of 91.94%, positive class precision of 95%, and positive class recall of 91%. The overall average expressed in terms of Mean Absolute Error and Mean Squared Error also proved the accuracy of the model used. Hence these findings indicate that this linear regression model is adequate for this binary classification problem and reasonably accurate in giving estimates.

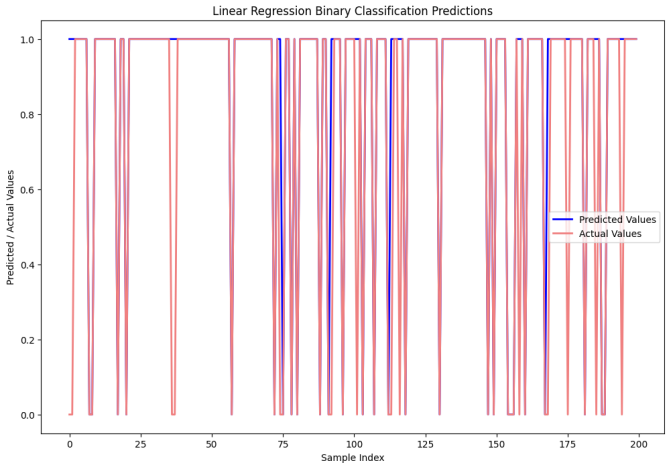

Figure 12. Linear Regression Binary Classification Predictions

B. *Logistic Regression*

- **Justification** : Logistic regression provides good results on this kind of datasets as logistic regression specifically designed for binary classification tasks handles both continuous and categorical features well, and can be easily regularized to prevent over fitting. The typical output of logistic regression is a probability of attack that are constrained between 0 and 1, which makes it much more interpretable in the context of intrusion detection it captures the log-odds relationship between network traffic attributes and the target variable, which is well-suited for modeling binary outcomes like intrusion detection
- **Training Details** : To improve the convergence of the model, the max_iter=5000 parameter was used and random_state=123 for reproducibility. In this case, the features of the training dataset were used to train the model. X_train_binary and labels. To obtain y_train_binary, fit method is used.
- **Prediction** : As a next step the seven fold cross validation was done and the predictions were made on the test data set. It classified the document according to whether the predicted probability was above or below 0.5 class label.
- **Hyperparameters** : Hyperparameter tuning includes the solver, a parameter that is specifically set to its default value of 'lbfgs' for optimization, and an L2 parameter to minimize overfitting. The term 'C' is used to determine the strength of the regularization; the smaller the value of 'C', the more it regularizes the model.
- **Evaluation and Tuning** : Hyperparameters were tuned by GridSearchCV which means that the function searches over the parameter space and uses cross- validation to assess the performance. Inside this parameter grid, we had C, which is the inverse of regularization strength with choices of [0.01, 0.1, 1, 10] and the penalty given as 'l2'. The grid search was built with a scoring parameter set to accuracy, the cross-validation set at 5-fold, and n_jobs=-1 which uses all of the available cores for the search parameter. Such approach made it possible to determine the best model parameters to enhance the performance on the test set.
- **Insights** : The plot in the figure above visually accredits high performance of the developed model. A good fit is achieved, where events that are similar in the number of either positive or negative are clustered close together. In quantitative terms, the given model yields an accuracy of 91.49%, for the positive class it gives a precision of 92% and for a positive class recall of 98%. The low Mean Absolute Error and Mean Squared Error again validated the amount of accuracy the model provided. By applying this model, the performance of the logistic regression in this binary classification is good, thus the model's prediction results are accurate.

C. *K-Nearest Neighbors (KNN)*

- **Justification** : K-Nearest Neighbors (KNN) Classifier is suitable for the dataset and seems to perform best due to its basic model which makes it easy to implement to capture complex structure without having to model the structure of the data or indeed the distribution of the dataset has mix of continuous and categorical data which makes it fit for its model as it can handle them. KNN

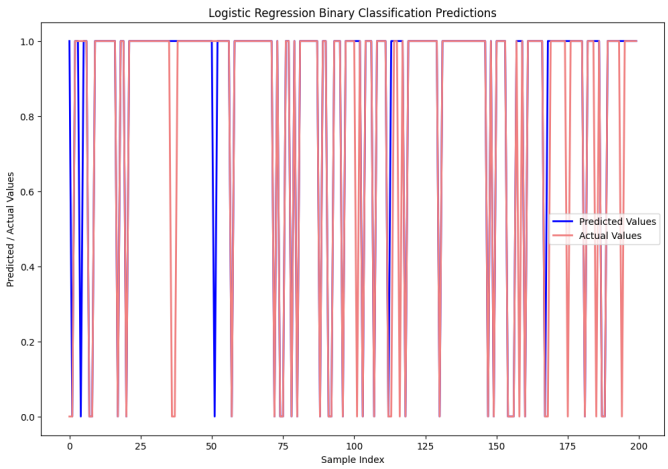

Figure 13. Logistic Regression Binary Classification Predictions

operates by putting a given data point in the class shared by the closest K neighbors of the feature space, and so is effective at discovering previously unrecognized patterns in traffic across a network

- **Training Details**: For training, the KNeighborsClassifier was set using n_neighbors=5 and, in classification of a test point, the model takes the nearest 5 neighbors. Using the fit method, the model was trained with the X_train_binary (input features) and the y_train_binary (labels).
- **Prediction** : After training, the forecast is made about the test data X_test_binary. The model measures the distance between the point of test and the training set, identifies the k closest samples and the class with the biggest share.
- **Evaluation and Tuning** : The K-Nearest Neighbors (KNN) model was tuned with GridSearchCV in order to find the best values of hyperparameters. A parameter grid was defined to tune the K-neighbors model n_neighbors with values [5, 7, 9] and the weight function used in prediction with options uniform and distance. GridSearchCV utilities was set with accuracy as the scoring parameter, cross valitaion with 5 folds and never parallelize using cv =5 and n_jobs=-1. First, the model was fitted to the training data, and then, the optimal values of the parameters the best values of features were chosen according to the results of the cross-validation, and then the model was used to predict outcomes on the test set.
- **Insights** : This plot tends to visually support the model concerning its efficiency in terms of structured and unstructured data. It also reveals that the actual values are nearly perfectly classified and close to the predicted values. Even from the quantitative perspective, the model yields a phenomenal generality of 99.95% and minimal error rates are observed. Based on these results, one can conclude that the ensure model of KNN is indeed a very successful representation for this particular binary classification problem and will produce exceptionally good prediction.

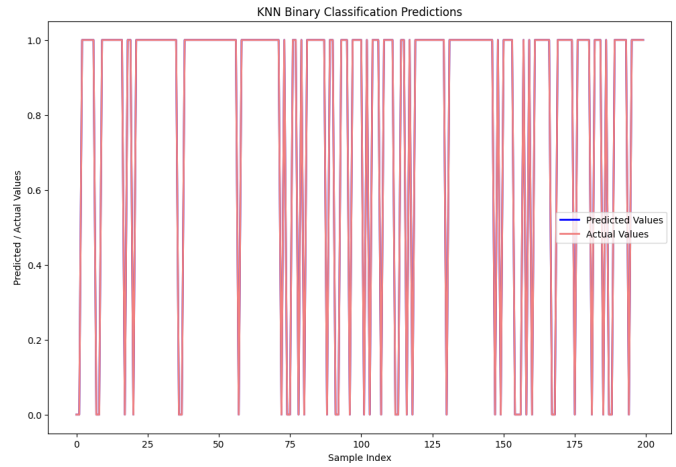

Figure 14. K Nearest Neighbor Binary Classification Predictions

D. *Linear Support Vector Machine (SVM)*

- **Justification** : Linear Support Vector Machine (SVM) becomes a excellent for this dataset as it is highly effective at separating data into distinct classes, such as normal vs. attack traffic. Linear kernel in SVM is computationally efficient and well-suited for this dataset where classes are linearly separable, such as distinguishing between normal and abnormal network traffic patterns also performs well when there is class imbalance as where normal traffic typically outnumbers attacks and SVM's ability to generalize well and handle high-dimensional data efficiently.
- **Training Details**: For this classification problem the binary classification model linear SVC with gamma of 'auto' was used. The kernel setting as 'linear' means the model has to try and place the classes in a linear hyperplane solution. The gamma parameter when set to 'auto', predicts the value of kernel coefficient for the linear kernel. It was then'fitted'by means of the fit method on the X_train_binary which are the input features and the y_train_binary which are the labels.
- **Prediction** : Following the training of the new model, the class of test samples from the feature vector X_test_binary is estimated. Thus by applying the decision boundary on the test data the test point is classified into one of the two classes, Zero or one, depending on which side of the hyhperplane it falls on.
- **Evaluation and Tuning** : That is why linear models including linear SVM models work well if the data is linearly separable or nearly so. It is possible to regulate such hyperparameters as C (regularization parameter) in order to may the gap between the optimal margin and the classification errors. In addition, gamma determines the impact of each support vector on the space, which

determines the complexity of the decision boundary. The binary classification metrics which are normally used include accuracy, precision, recall and F1-score.

- **Insights** : The plot verifyingly reflects that the model is perfectly suitable and gives a good result. By the aid of the predicted/tested equation, (t) values equal to actual values, which is an implication of a correct classification. Measuring the proposed model performance in quantitative ways, it has a high value of the accuracy ratio, equal to 92.52%, the strong positive class precision, which is 95%, and the high positive class recall value of 92%. The Mean Absolute Error and Mean Squared Error values shown also testify for the model's precision. These results imply that the Linear SVM model is accurate and effective for this binary classification problem to give reasonable predictions.

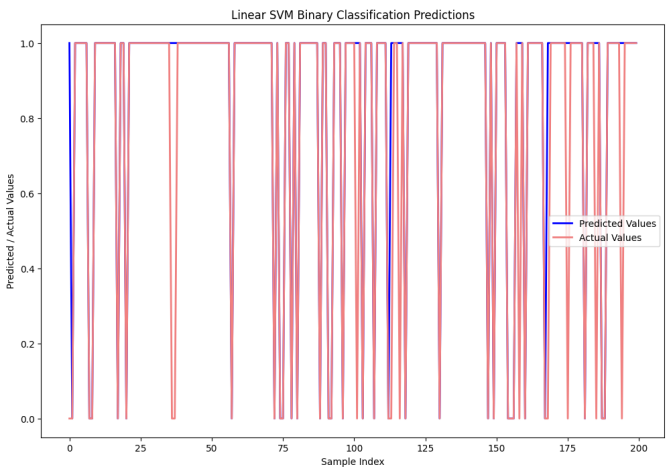

Figure 15. Support Vector Machine Binary Classification Predictions

E. *Naive Bayes*

- **Justification** : Naive Bayes is based on Bayes' Theorem and the model is simple and efficient and can handle different types data in the dataset. Naive Bayes also works well with high-dimensional data, which is common in network traffic, and can classify network traffic as normal or various attack types, such as DoS or Fuzzers. while we compare to other models this model is less prone to over fitting.Naive Bayes can also handle class imbalance by adjusting class priors.
- **Training Details** : For this binary classification task the Naïve Bayes Algorithm of Gaussian type, GaussianNB() was employed. It was fitted on the feature set X_train_binary and labels y_train_binary of NCBI datasets. GaussianNB classifier calculates the mean, and variance of the Gaussian required for each class and feature.
- **Prediction** : When the method training was complete, the predict method was ran on X_test_binary in order get the predictions. In the case of the test set, for every data instance, the model computes the probability of data belonging to the particular class 0 or 1, according to the learned mean and variance of Gaussians. The class with the highest probability is regarded as the predicted label of the instance.
- **Evaluation and Tuning** : Despite the fact the Naive Bayes classifier is simpler, it is preferred to be used when working with large datasets as well as with the limited computational capabilities. But it has limitations in that it assumes the features are independent and derived from a Gaussian distribution – an assumption often untrue in practice datasets. Nonetheless, it can do fairly well, particularly where the assumption of independence is quite reasonable. From the table we can see that performance can be assessed using parameters such as accuracy, precision, recall, and F1 score.
- **Insights** : The plot helps to visually corroborate a high effectiveness of the proposed model. The observation here is that the predicted values follow those of the actual values very keenly and hence people would be right in their classification. For the quantitative aspect, overall accuracy stands at 92.72%, the testing with a positive label yield a precision of 96% with an overall recall of 91% for the positive class. The low Mean Absolute Error and Mean Squared Error back the reality of the model that has been developed. Consequently, the findings prelude that the Naive Bayes model is appropriate and accurate for this binary classification problem.

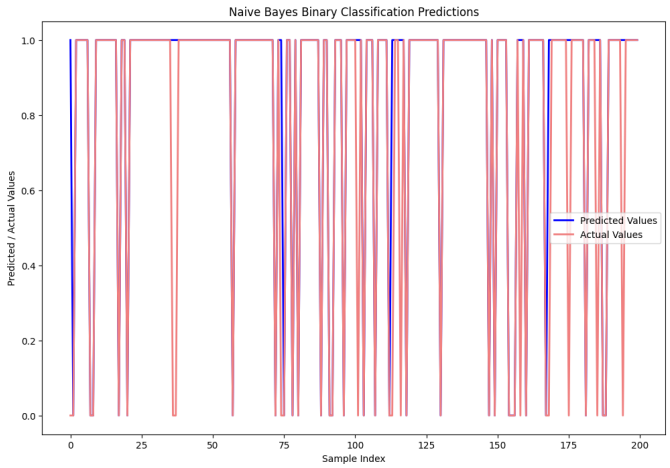

Figure 16. Naive Bayes Binary Classification Predictions

F. *Gradient Boosting*

- **Justification** : We choose Gradient Boosting because of its ability to handle complex, non-linear relationships between features and the target variable, by building multiple decision trees sequentially, where each tree attempts to correct the errors made by the previous one as the dataset having hight number of records and high dimensional features Gradient Boosting can effectively capture intricate patterns in the data and identify subtle

distinctions between normal traffic and various attack types. It also handles both continuous and categorical features well making it one of the best model to be used for the dataset.

- **Training Details** : The Gradient Boosting Classifier was defined with the random_state parameter set to 123 for getting the same results with any run The trained model was created by calling on fit function on Gradient Boosting Classifier with X_train and y_train as inputs. The model learns by coming up with residual errors that are minimized in the trees.
- **Prediction** : On the test set (X_test) the predict method was utilized to generate some predictions. The final decision or output is an aggregation of all the trees in the decision tree with the element of percentage votes importance.
- **Evaluation and Tuning** : Hyper tuning parameters such as learning rate, number of decision trees (n_estimators) and depth of a tree (max_depth) are other parameters that need to be set to get the best result as well as to solve the problem of overfitting. The measure usually applied is accuracy, precision, recall, and F1-score or evaluation.
- **Insights** : In the plot, one can visually ascertain that the model fits the data well, hence it performs transactions above an exceptional level. The predicted values fall nearly in line with the actual values, which bears evidence to correct classification. In a quantitative way, the results are accuracy as 97.96%, positive class precision as 99%, positive class recall as 97%. The Mean Absolute Error and Mean Squared Error further corroborate the results of the current model. It also shows that Gradient Boosting is appropriate for this binary classification problem and its predictions are fairly accurate.

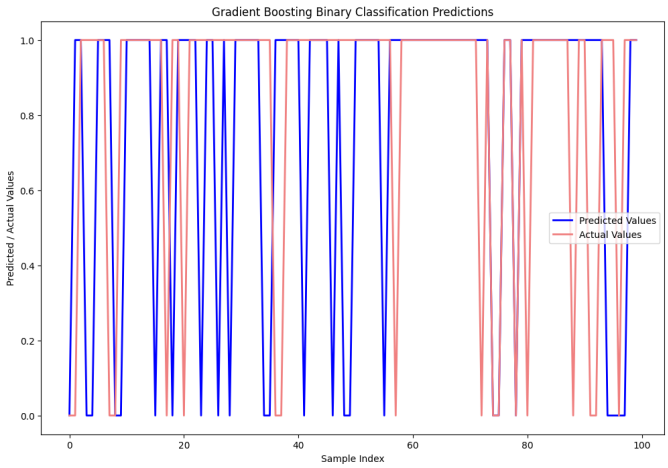

Figure 17. Gradient Binary Classification Predictions

G. *Random Forest*

- **Justification** : We use Random Forest as its ability to handle large datasets with high-dimensional features, such as the 45 attributes in this dataset.Random Forest handles both categorical and continuous data efficiently, which is important for the diverse nature of network traffic. It also provides valuable feature importance metrics, helping to identify which network attributes contribute most to detecting attacks, such as packet counts or flow duration it is resistant to noisy data, which is common in network intrusion detection tasks, and can improve classification accuracy even with class imbalance making it flexible and a good choice
- **Training Details** : The Random Forest Classifier model was also created using n_estimators=100 because it generates 100 trees and random_state= 42 since it creates reproducible models. The model was build using fit method available with the nomenclature training data set (X_train and y_train) each tree learns the decision rule randomly with subset of features and data points available.
- **Prediction** : After training the model, in the test set (X_test) the predict method was applied to obtain the predictions. The last decision which is given by the random forest model is the sum of the trees in the forest.
- **Evaluation and Tuning** : Meaning it is composed of hyperparameters like n_estimators (number of decision trees), max_depth (the depth of decision trees), and max_features (maximum numbers of features to consider when splitting data). It is common to assess performance mainly by means of accuracy, precision, recall and F1 score.
- **Insights** : The plot above gives a visible affirmation of how well the model scores. The values estimated are hence very close to the actual values to highlight correct classification. From a quantitative point of view, the proposed architecture reaches 98.89% of accuracy, 99% of precision for the positive class, and a 99% of recall for the positive class. This is supported by the Mean Absolute Error (MAE) and Mean Squared Error (MSE), of which the first is 2.2, and the second, 4. In this study, the Random Forest model was determined to be suitable for predicting the binary outcome of the dependent variable.

VIII. RESULT COMPARISON

In this section, we compare the performance of various machine learning models applied to detect network intrusions in our Intrusion Detection System (IDS). We evaluate these models based on multiple metrics, including Accuracy, R^2 score, Precision, Recall, F1-Score, and Mean Absolute Error (MAE). These key metrics provide a comprehensive view of the model's ability to correctly identify malicious and normal network activities. The performance of each model is summarized in tables and visualized in graphs for better clarity and comparison.

A. *Comparison Metrics*

1) Accuracy: Accuracy remains one of the popular measure in classification tasks. It quantifies the total total correct

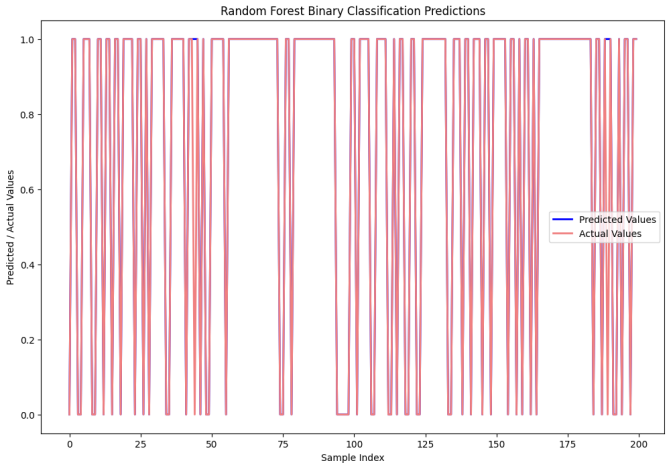

Figure 18. Random forest Binary Classification Predictions

predictions done by the model. In the case of our Intrusion Detection System, accuracy provides us with a big picture of the percentage of times the model identifies the network traffic as either Normal or Malicious. In this IDS(intrusion detection system) accuracy helps get an idea of how this model makes accurate predictions.

Inference: As shown in the accuracy comparison plot (Fig 19), all models achieved strong results with K-nearest Neighbor KNN model having the highest accuracy of 99.95% while Random Forest closely followed with 98.89% then Gradient Boosting model with 97.95%. These models performed better than the other models especially when handling more complex data . Logistic Regression was seen to be the least accurate with a score of 91.50% thus pointing out to a fact that this model was more challenged in this aspect of classification than the other complex models. In general, the results of most models were favourable though the KNN emerged as the best.

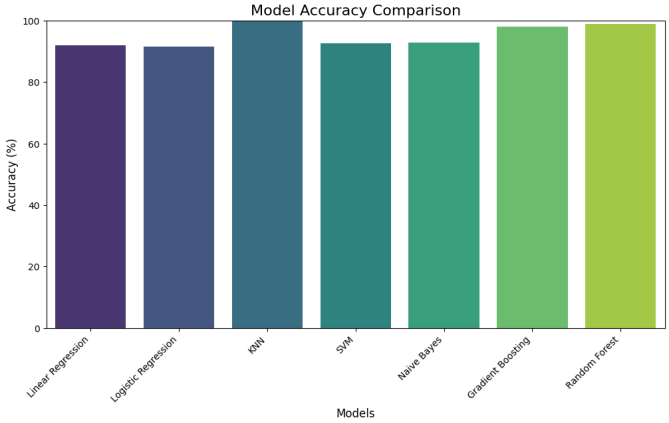

Figure 19. Accuracy Comparison for all models

2) R^2 Score: The R^2 score in our Intrusion Detection System (IDS) helps us understand how much of the variance in network traffic (normal or malicious) can be predicted by our model. A score closer to 1 shows that the model is good at detecting malicious activities, explaining most of the patterns in the data. On the other hand, a lower R^2 means the model might not be capturing the differences between normal and malicious activities well. In our case, a high R^2 score indicates the IDS can effectively spot attacks while minimizing errors. If the score is low, we might need to look into improving the features or the model's complexity.

Inference: As shown in the R^2 comparison plot (Fig 20), where K-Nearest Neighbors (KNN) had the highest value of R^2 equal 99.72%, secondly Random Forest 94.92% and third Gradient boosting 90.72% outperforming other models. Linear Regression model had the lowest R^2 score equal to 56.11% which indicating that it was less effective at capturing complex relationships in the data compared to other models.

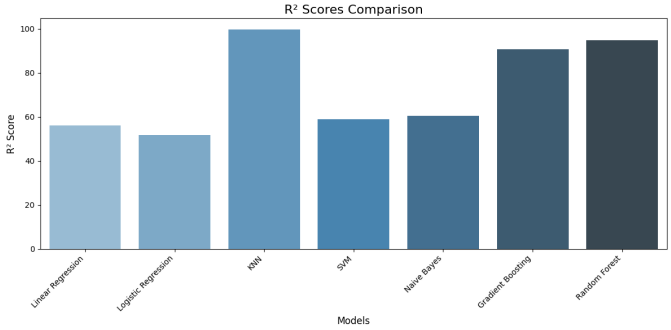

Figure 20. R^2 Comparison for all models

3) Precision: In this intrusion detection system IDS the key aim was to reduce false positive rates to a minimum and they mean that normal traffic is marked as an attack. Recall informs us of the proportion of cases the model correctly identifies when it labeled an activity as Malicious. In simpler terms, it answers the question: "The percentage of all the activities identified as malicious, how many were actually malicious?"

Inference: The Precision values show that K-Nearest Neighbors (KNN) achieved a perfect precision of 1.00, meaning it correctly identified almost all malicious traffic without falsely labeling normal traffic. Random Forest also performed very well with a precision of 0.99, and Gradient Boosting had a precision of 0.98. However, Logistic Regression had a slightly lower precision of 0.91, indicating more false positives. Overall, KNN was the top performer, but the other models also showed strong precision values.

4) Recall: Recall is all about detecting as many Malicious activities as possible. In an IDS, it's more important to not miss any malicious activities than to make the occasional false alarm. Recall tells us the proportion of actual malicious activities that were correctly identified by the system.

Inference: For Recall, K-Nearest Neighbors (KNN) again stood out with a perfect recall of 1.00, meaning it detected all malicious instances. Random Forest followed closely with a recall of 0.99, and Gradient Boosting had a strong recall of 0.98. However, Linear Regression had a lower recall of 0.92, meaning it missed more malicious instances compared to the

other models. All models showed good recall, with KNN being the best at identifying both normal and malicious traffic.

5) F1 score: The F1-Score is a combination of precision and recall, and it's especially useful when we want a balance between detecting malicious activities and minimizing false alarms. The F1-score is important because in an IDS, we don't want to sacrifice too much precision for recall, or vice versa.

Inference: When looking at the F1-Score, K-Nearest Neighbors (KNN) had the highest score of 1.00, showing the best balance between precision and recall. Random Forest and Gradient Boosting followed with high F1-scores of 0.99 and 0.98, respectively. Linear Regression had a lower F1-score of 0.91, suggesting a trade-off between precision and recall. Overall, KNN was the best at maintaining a perfect balance between the two metrics.

6) Mean Absolute Error: MAE(Mean Absolute Error) is very easy to compute and can be used to identify the average difference between the forecasted and real values. In our IDS, we also used it to determine how far the model's prediction is from the actual labels. This tells us the average error values in terms of absolute differences between our predictions and actual values.

Inference: The Mean Squared Error (MSE) values show that K-Nearest Neighbors (KNN) had the lowest error, with a value of 0.00048, indicating that the model was able to make very accurate predictions with very small deviations. Random Forest came next with an MSE of 0.01109, followed by Gradient Boosting at 0.02042. These values indicate that the ensemble models performed better in reducing the error compared to the other models. On the other hand, Linear Regression had a higher MSE of 0.08062, suggesting that it was not as effective in minimizing the prediction error, which may be due to its inability to capture more complex patterns in the data. Overall, models like KNN and Random Forest achieved much lower errors than the others, making them more reliable for accurate predictions.

7) Mean Squared Error: MSE measures the average squared difference between the predicted and actual values. Lower MSE values indicate that the model predictions are closer to the actual data. MSE takes the errors and squares them, which means it penalizes larger errors more than MAE. This is useful in our IDS because it helps the model focus on minimizing large misclassifications. A smaller MSE indicates that the model's predictions are closer to the actual values.

Inference: The Mean Squared Error (MSE) plot shows that K-Nearest Neighbors (KNN) achieved the lowest MSE of 0.00048, indicating that it was the most accurate in minimizing the prediction error. Random Forest followed with an MSE of 0.01109, and Gradient Boosting had a value of 0.02042, both indicating that these models performed quite well in reducing error. Linear Regression, on the other hand, had the highest MSE of 0.08062, meaning it was less accurate at predicting compared to the other models. The plot clearly highlights that KNN outperforms the others with the lowest error, while the other models, especially ensemble ones, also performed well in minimizing the MSE.

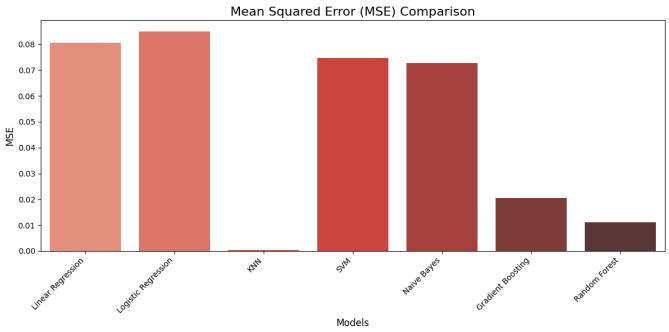

Figure 21. Mean Squared Error Comparison for all models

8) Root Mean Square Error: RMSE is just the square root of MSE, and it brings the error measure back into the same units as the target variable. This makes it easier to interpret and compare with the actual data. For our IDS, RMSE helps us assess the average error in prediction terms that are more understandable.

Inference: The Root Mean Squared Error (RMSE) results show that K-Nearest Neighbors (KNN) had the lowest RMSE of 0.02197, meaning it had the smallest overall error between predicted and actual values. Random Forest had an RMSE of 0.10532, and Gradient Boosting was at 0.14288, showing that these models also performed well but with slightly higher errors compared to KNN. Linear Regression, on the other hand, had the highest RMSE of 0.28394, indicating it had a larger difference between its predictions and the actual values. Overall, KNN performed the best with the lowest RMSE, followed by the ensemble models, and Linear Regression had the most significant error.

IX. CONCLUSION

In conclusion, it can be observed , that the K- Nearest Neighbors (KNN) surpassed other models in the network intrusion detection as it recorded higher accuracy, precision, recall, and F1-score than other models in identifying malicious traffic. Similarly, Random Forest and Gradient Boosting Machine also revealed good performance, mainly in terms of error, but KNN was the best. Linear Regression and Logistic Regression models had comparatively lower accuracy and recall rates, showing the difficulty of the models in solving the complicated intrusion detection problems. Still KNN and the ensemble models like Random Forest and Gradient Boosting was found to be the most efficient and accurate solutions to use in detecting intrusions in a network.

X. REFERENCES

REFERENCES

[1] D. Wells, "UNSW-NB15 Dataset," Available at: https://www.kaggle.com/datasets/mrwellsdavid/unsw-nb15

[2] NIST, "Exploratory Data Analysis," Available at: https://www.itl.nist.gov/div898/handbook/eda/section1/eda11.htm

[3] J. Osborne, *Best Practices in Data Cleaning: A Complete Guide to Everything You Need to Do Before and After Collecting Your Data*, SAGE Publications, 2012.

[4] I. Ilyas and X. Chu, *Data Cleaning*, ACM Press, 2021.

[5] A. Zheng and A. Casari, *Feature Engineering for Machine Learning: Principles and Techniques for Data Scientists*, O'Reilly Media, 2018.
[6] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, and E. Duchesnay, "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825-2830, 2011. Available at: https://scikit-learn.org/stable/
[7] W. S. Cleveland, *Visualizing Data*, Hobart Press, 1993. Available at: https://matplotlib.org/stable/gallery/index.html
[8] Folium, "Python Data, Leaflet.js Maps," Available at: https://python-visualization.github.io/folium/
[9] Kumar, Sathish P., and Arun Raaza. "Study and analysis of intrusion detection system using random forest and linear regression." Periodicals of Engineering and Natural Sciences (PEN) 6, no. 1 (2018): 197-200.
[10] Subba, Basant, Santosh Biswas, and Sushanta Karmakar. "Intrusion detection systems using linear discriminant analysis and logistic regression." In 2015 Annual IEEE India Conference (INDICON), pp. 1-6. IEEE, 2015.
[11] Liao, Yihua, and V. Rao Vemuri. "Use of k-nearest neighbor classifier for intrusion detection." Computers security 21, no. 5 (2002): 439-448.
[12] Lin, Wei-Chao, Shih-Wen Ke, and Chih-Fong Tsai. "CANN: An intrusion detection system based on combining cluster centers and nearest neighbors." Knowledge-based systems 78 (2015): 13-21.
[13] Peddabachigari, Sandhya, Ajith Abraham, Crina Grosan, and Johnson Thomas. "Modeling intrusion detection system using hybrid intelligent systems." Journal of network and computer applications 30, no. 1 (2007): 114-132.
[14] Jha, Jayshree, and Leena Ragha. "Intrusion detection system using support vector machine." International Journal of Applied Information Systems (IJAIS) 3 (2013): 25-30.
[15] Krishnaveni, S., Palani Vigneshwar, S. Kishore, B. Jothi, and S. Sivamohan. "Anomaly-based intrusion detection system using support vector machine." In Artificial intelligence and evolutionary computations in engineering systems, pp. 723-731. Singapore: Springer Singapore, 2020.
[16] Panda, Mrutyunjaya, and Manas Ranjan Patra. "Network intrusion detection using naive bayes." International journal of computer science and network security 7, no. 12 (2007): 258-263.
[17] Sharmila, B. S., and Rohini Nagapadma. "Intrusion detection system using naive bayes algorithm." In 2019 IEEE International WIE Conference on Electrical and Computer Engineering (WIECON-ECE), pp. 1-4. IEEE, 2019.
[18] Esmaily, Jamal, Reza Moradinezhad, and Jamal Ghasemi. "Intrusion detection system based on multi-layer perceptron neural networks and decision tree." In 2015 7th Conference on Information and Knowledge Technology (IKT), pp. 1-5. IEEE, 2015.
[19] Anzer, Ayesha, and Mourad Elhadef. "A multilayer perceptron-based distributed intrusion detection system for internet of vehicles." In 2018 IEEE 4th international conference on collaboration and internet computing (CIC), pp. 438-445. IEEE, 2018.
[20] Musa, Usman Shuaibu, Megha Chhabra, Aniso Ali, and Mandeep Kaur. "Intrusion detection system using machine learning techniques: A review." In 2020 international conference on smart electronics and communication (ICOSEC), pp. 149-155. IEEE, 2020.
[21] Ashiku, Lirim, and Cihan Dagli. "Network intrusion detection system using deep learning." Procedia Computer Science 185 (2021): 239-247.